UNIVERSIDADE ESTADUAL DO PIAUÍ

Campus Piripiri · Curso de Computação

SEGURANÇA COMPUTACIONAL / 2026

Auditoria estática de uma TV box Android

RELATÓRIO TÉCNICO · GRUPO 1

Maria Rita e Pedro Gabryel

Aquisições: 10 de outubro de 2026 (data informada pelos autores)

Unidade: 14f2782704d344000052

94	49	9
APKs inventariados	arquivos init	achados preliminares

Composição, privilégios, inicialização e atualização: evidências estáticas para avaliar o possível reaproveitamento educacional.

Piripiri · 2026

Resumo

Este trabalho examinou uma TV box Android por aquisição lógica via ADB e análise estática dos arquivos obtidos. O objetivo foi mapear sua composição e identificar riscos e componentes que mereçam investigação para eventual reaproveitamento educacional, conforme a proposta da disciplina. A coleta principal preservou 507 arquivos registrados por SHA-256, incluindo 94 APKs. Foram analisados os 94 manifestos, com 1272 declarações de permissões e 2129 componentes; também foram mapeados 49 arquivos init, 71 serviços e 793 ações/imports. Uma aquisição complementar obteve três scripts de inicialização e documentou cinco ausências nos caminhos pesquisados.

O principal resultado técnico é a reconstrução do fluxo do par UOTA: configuração e atualização por HTTP, download definido pela resposta remota, comparação MD5 e caminhos condicionais de instalação silenciosa. O cliente também contém validação TLS permissiva e uma modalidade de execução de comandos. A análise esclareceu que as consultas a Google/Baidu usam o cabeçalho HTTP Date como fonte de hora e confirmou a lista NTP embutida. No qcode, foi identificada rotina potencial de exclusão de arquivos grandes associada ao boot. A lógica funcional da GetApps permanece parcialmente desconhecida por proteção nativa; a finalidade do serviço root qw também não foi determinada.

A matriz reúne nove achados preliminares: quatro configurações vulneráveis/mal configuradas, três capacidades potencialmente inseguras e dois componentes desconhecidos. Nenhum componente recebeu a classe malicioso/indesejado comprovado. Os resultados descrevem uma única unidade e sua variante de firmware; não demonstram exploração, entrega de payload hostil, capacidade física de RAM ou segurança operacional para uso institucional.

Palavras-chave: Android; TV box; análise estática; atualização OTA; preservação de evidências.

Sumário

1. Objetivo, perguntas e limites do escopo 4

2. Unidade e caracterização técnica 5

3. Método, aquisições e preservação 7

4. Inventário, permissões e componentes 10

5. Inicialização e provisionamento 12

6. Seleção dos aplicativos prioritários 14

7. Reconstrução técnica dos aplicativos 15

8. URLs, indicadores e cobertura da pesquisa 20

9. Correlação com evidências dinâmicas recebidas 22

10. Matriz de achados e classificação 23

11. Recomendações decorrentes da análise 25

12. Limitações e resultados inconclusivos 26

13. Conclusão 27

Apêndice A — Catálogo de evidências e hashes representativos 28

Apêndice B — Reprodução e organização dos anexos 30

Apêndice C — Entregáveis do Grupo 1 31

Apêndice D — Referências 32

1. Objetivo, perguntas e limites do escopo

O objetivo do Grupo 1 é mapear a composição do firmware e identificar componentes que mereçam investigação adicional, conforme as seções 8.1–8.3 da proposta. A procedência por apreensão não foi tratada como evidência de comportamento malicioso.

A investigação responde a quatro perguntas: quais pacotes e privilégios existem; quais mecanismos podem iniciar ou manter componentes; como o equipamento pode receber, baixar e instalar conteúdo; e quais capacidades têm evidências suficientes para classificação preliminar. O inventário cobre todos os pacotes registrados na coleta. A reconstrução aprofundada do código concentra-se em quatro aplicativos selecionados a partir dos comportamentos observados pelo Grupo 2.

O procedimento produz uma aquisição lógica de arquivos e metadados com o Android em execução. Não produz imagem integral das partições nem uma auditoria completa de todos os binários nativos. O relatório distingue declarações de configuração, capacidades reconstruídas do código, observações dinâmicas recebidas e hipóteses. Uma função presente no APK pode depender de condições que não ocorreram; uma string pode nunca ser usada.

2. Unidade e caracterização técnica

2.1 Identificação e histórico da amostra

A amostra é uma unidade física, compartilhada com o Grupo 2, conforme confirmação dos autores. As coletas dos dois grupos representam estados da mesma box, e não duas amostras independentes. O serial ADB, a fingerprint e o conjunto de pacotes/caminhos permitem vincular os arquivos analisados à unidade identificada.

A proposta orienta unidades próprias por grupo; o uso compartilhado é registrado como limitação de amostragem e de controle do estado inicial. O histórico recebido registra uso de aplicativos, geração de caches/dados e concessão seguida de revogação de permissão de armazenamento da GetApps. A unidade não é apresentada como referência intocada de fábrica. O registro detalhado está em evidencias/REGISTRO_ALTERACOES_NA_UNIDADE.md.

Não foi disponibilizado registro fotográfico ou transcrição da etiqueta da carcaça. “TV BOX” é o modelo declarado pelo software. Isso limita a identificação comercial aparente, mas não impede vincular esta aquisição à amostra por seus identificadores técnicos. Não foram caracterizadas outras unidades ou clusters das aproximadamente 200 boxes do lote.

2.2 Hardware e software declarados

Item	Resultado e limite	Evidência
Modelo/fabricante declarados	TV BOX / Allwinner	getprop.txt, propriedades ro.product
Placa/dispositivo	dolphin-fvd-p1; hardware sun8iw7 / sun8iw7p1	getprop.txt e kernel_cpu.txt
SoC	Allwinner H3 segundo o repasse do Grupo 2; hardware/ABI coletados são compatíveis com a caracterização reportada, sem verificação física adicional	referencias/grupo2/Repasse_Grupo1.md; kernel_cpu.txt
CPU/ABI	ARMv7, armeabi-v7a; dois processadores listados no snapshot. Não equivale à contagem física definitiva de núcleos	kernel_cpu.txt
RAM	MemTotal declara 8388608 kB (8 GiB); capacidade física não validada	memoria_armazenamento.txt
Armazenamento	Filesystem /system: 2031440 blocos de 1 KiB (~1,94 GiB); /data: 4371984 (~4,17 GiB). São filesystems, não a capacidade total física da flash	memoria_armazenamento.txt
Android	Release declara 11.1; API declara 24; fingerprint contém 7.0	getprop.txt:149,162,163
Kernel	Linux 4.4.55, ARMv7, build informado de dezembro de 2020	kernel_cpu.txt:1
Patch de segurança	2016-11-05 declarado. Não verifica correções efetivas nem demonstra uma CVE específica	getprop.txt:164

Item	Resultado e limite	Evidência
Build	Allwinner/dolphin_fvd_p1/dolphin-fvd-p1:7.0/NRD91N/20201214173318:eng/test-keys	getprop.txt:149
Controles de acesso	ADB shell UID 0; SELinux Permissive; ro.adb.secure=0, ro.secure=0, ro.debuggable=1	kernel_cpu.txt; getprop.txt

As estatísticas de memória, incluindo CommitLimit e o tamanho de tmpfs, exigem cautela ao interpretar o MemTotal; não foi efetuado teste de RAM física.

2.3 Interpretação dos metadados

O rótulo Android 11.1 conflita com a API 24 e com a fingerprint Android 7.0. A caracterização usada neste relatório é a plataforma que declara API 24, compatível com Android 7.0 segundo a documentação oficial [R1]. A data declarada de patch identifica uma configuração antiga; não comprova, isoladamente, uma vulnerabilidade CVE específica ou a ausência de toda correção posterior.

A RAM física não foi medida. O relatório recebido atribui a discrepância a falsificação; a evidência própria permite afirmar que MemTotal declara 8 GiB e que outras estatísticas exigem cautela, sem estabelecer aqui a capacidade física correta ou a intenção de quem alterou o valor. Da mesma forma, os tamanhos de /system e /data correspondem a filesystems, não ao total da flash.

As listas de pacotes/caminhos dos dois grupos coincidem como conjunto. Não há hashes anteriores de todos os APKs no repasse para demonstrar equivalência byte a byte entre os estados. Portanto, a comparação sustenta a continuidade da composição registrada, com esse limite.

3. Método, aquisições e preservação

3.1 Sequência de análise

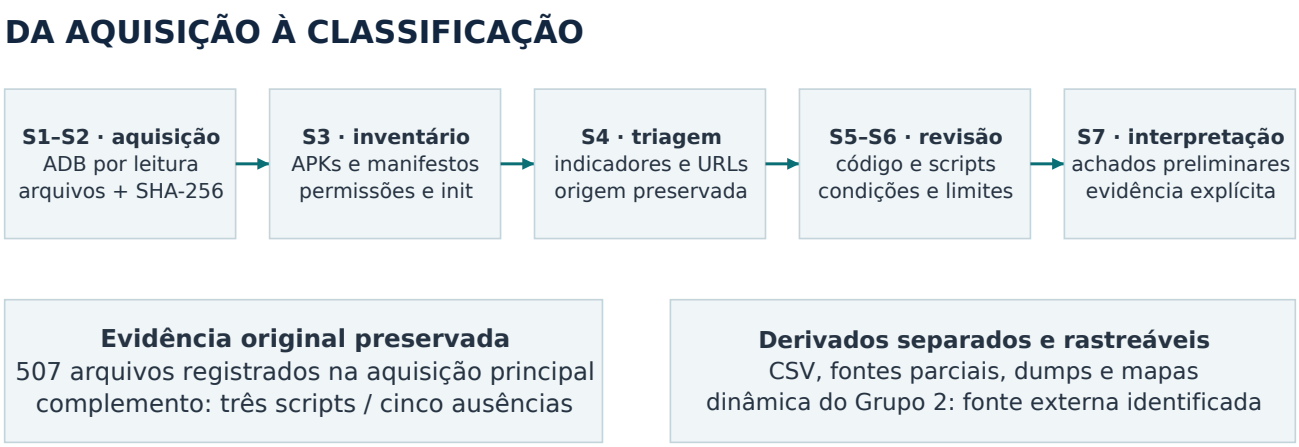


Figura 1. A aquisição é preservada separadamente dos resultados derivados. A classificação usa evidências estruturais e leitura de código, com correlação dinâmica identificada como fonte externa.

Etapa	Procedimento	Produto verificável
S1 — caracterização	Leitura de getprop, UID/SELinux, kernel, memória, armazenamento e pacotes	Arquivos em sistema/ e metadados da aquisição
S2 — aquisição	Cópia por ADB dos APKs, diretórios associados, framework e init disponíveis	507 arquivos registrados, logs e SHA256SUMS
S3 — cobertura estrutural	aapt badging/xmltree, inventário, extração de certificados e bibliotecas	94 APKs e manifestos; CSVs de permissões/componentes
S4 — triagem	Busca guiada de indicadores e extração ampla de URLs candidatas	Indicadores com origem; URLs com membro/offset/codificação
S5 — reconstrução	JADX dos quatro APKs prioritários e revisão dos fluxos relevantes	Fontes parciais, logs e dumps de instruções preservados
S6 — inicialização	Mapeamento dos init e aquisição dos scripts referenciados	Serviços, ações/imports, três scripts e cinco ausências
S7 — interpretação	Cruzamento de código, metadados e material recebido do Grupo 2	Matriz de achados, limites, indicadores e relatório

As operações na box foram de leitura: getprop, cat, pm list, dumpsys e pull. Nenhum APK foi instalado, removido ou desativado pelo procedimento do Grupo 1; nenhum script coletado foi executado. A análise de código e a geração dos anexos ocorreram no computador de trabalho. O acesso foi USB/ADB no Ubuntu em WSL 2, com encaminhamento USB por usbipd.

Foi orientado manter a unidade sem Wi-Fi/Ethernet durante a aquisição. O Grupo 1 não produziu captura de rede independente para comprovar ausência de tráfego nessa etapa. As observações de rede discutidas na seção 9 pertencem ao material recebido do Grupo 2.

3.2 Aquisições e integridade

Conjunto	Identificador	Horário local registrado pelo host	Resultado
Principal	20261012_204535_l0sQMz	12/10/2026, 20:45:35–20:48:04	Zero falhas registradas; 507 arquivos com hashes verificados
Complementar	20261012_225942_ez2GjA	12/10/2026, a partir de 22:59:42	Três scripts obtidos; cinco caminhos ausentes; hashes registrados verificados

As listas de SHA-256 dos arquivos brutos foram preservadas. Fontes reconstruídas, manifestos e documentos têm conjuntos de hashes próprios. SHA-256 permite conferir a integridade relativa à referência registrada; não autentica a origem do firmware, não prova ausência de malware e não transforma uma aquisição lógica em imagem integral. A ausência de um arquivo também é um resultado de escopo limitado: caminho pesquisado, sistema montado e momento da coleta.

3.3 Ferramentas e limitações de reconstrução

Ferramenta	Versão registrada	Finalidade
ADB	34.0.4-debian / 1.0.41	Aquisição lógica e leitura de metadados
aapt	0.2-debian	Metadados e árvore dos manifestos
JADX	1.5.0	Reconstrução Java e dumps de instruções
Java	OpenJDK 21.0.12.1	Execução do decompilador
Python	3.12.3	Inventários e mapas estruturados
OpenSSL	3.0.13	Extração e descrição de certificados v1

O JADX produziu fontes parciais: foram encontrados 212 stubs “Method not decompiled” no UOTA principal, 178 no backup, 87 no qcode e 1 na GetApps, incluindo bibliotecas. Essas contagens medem ocorrências na saída, não a correção dos demais métodos. Retorno zero do decompilador não foi tratado como reconstrução completa.

O método de obtenção/validação do profile dos UOTA exigiu revisão manual do dump com comentários debug, preservado em analise/.../fallback/. O stub UnsupportedOperationException é gerado pelo decompilador e não foi interpretado como instrução original do aplicativo. Os trechos citados correspondem à execução documentada; regenerações podem mudar a numeração de linhas.

3.4 Critério de evidência

Tipo de resultado	O que sustenta	O que não sustenta sozinho
Configuração observada	Valor registrado de propriedade, UID, estado ou flag	Estado de toda a frota ou autenticidade dos metadados
Declaração estrutural	Permissão, receiver, serviço ou comando existente	Concessão, execução ou sucesso operacional

Tipo de resultado	O que sustenta	O que não sustenta sozinho
Capacidade reconstruída	Encadeamento estático de chamadas e condições	Que o caminho foi usado ou produziu dano
Correlação externa	Compatibilidade com captura/log do Grupo 2	Execução de partes não observadas no experimento
Negativo/inconclusivo	Limite do procedimento e questão ainda aberta	Ausência universal de comportamento ou segurança comprovada

4. Inventário, permissões e componentes

4.1 Cobertura estrutural

Os 94 APKs estão distribuídos por caminho registrado: 41 em /system/app, 48 em /system/priv-app, 4 em /data/app e 1 em /system/framework. Há 16 pacotes com UID 1000 e 49 com flag PRIVILEGED reportada; diretório, flag e UID não são sinônimos. Os dois UOTA estão em /system/app e compartilham android.uid.system, sem a flag PRIVILEGED observada em seu bloco.

Os manifestos contêm 988 activities, 60 activity-alias, 659 services, 325 receivers e 97 providers, totalizando 2129 declarações. Há 45 receivers com BOOT_COMPLETED em seus filtros. Esses números incluem componentes de SDKs e não significam que todos estejam ativos, exportados ou executando.

As permissões solicitadas no manifesto e as concedidas reportadas no dumpsys foram mantidas em fontes/colunas distintas. Permissões de shared UID não foram atribuídas como solicitações individuais a todos os seus pacotes. Valores de exported/enabled ausentes foram preservados como não declarados, sem aplicar um default de outra versão Android.

O inventário de bibliotecas possui 216 entradas, incluindo membros dos APKs, arquivos .so copiados e bibliotecas da GetApps em assets. Uma biblioteca pode aparecer nas duas origens; não se trata de 216 bibliotecas únicas. A finalidade interna de cada biblioteca não foi completamente reconstruída.

Anexos estruturados:

- inventário APKs e versão JSON;
- certificados extraídos;
- permissões solicitadas;
- componentes e filtros;
- bibliotecas nativas.

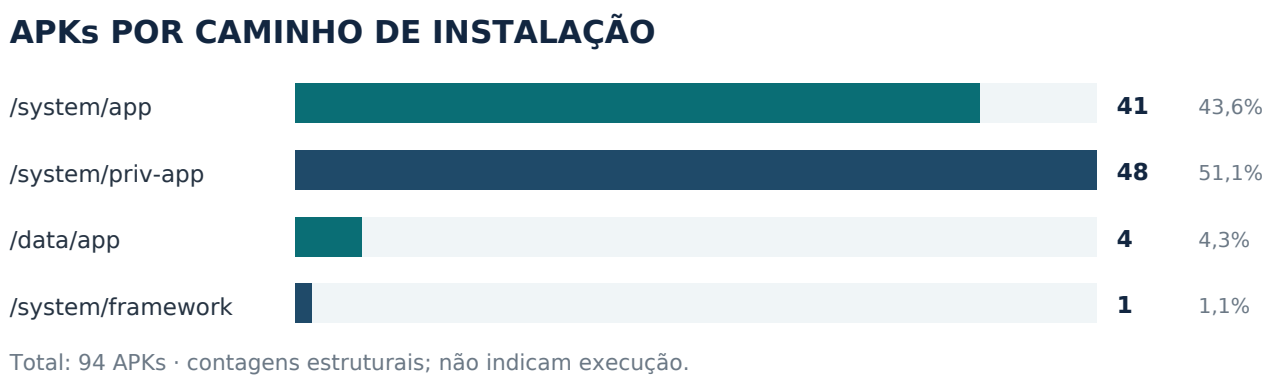


Figura 2. Distribuição dos 94 APKs pelo caminho registrado no gerenciador de pacotes. Diretório de instalação, UID e flag PRIVILEGED são atributos distintos.

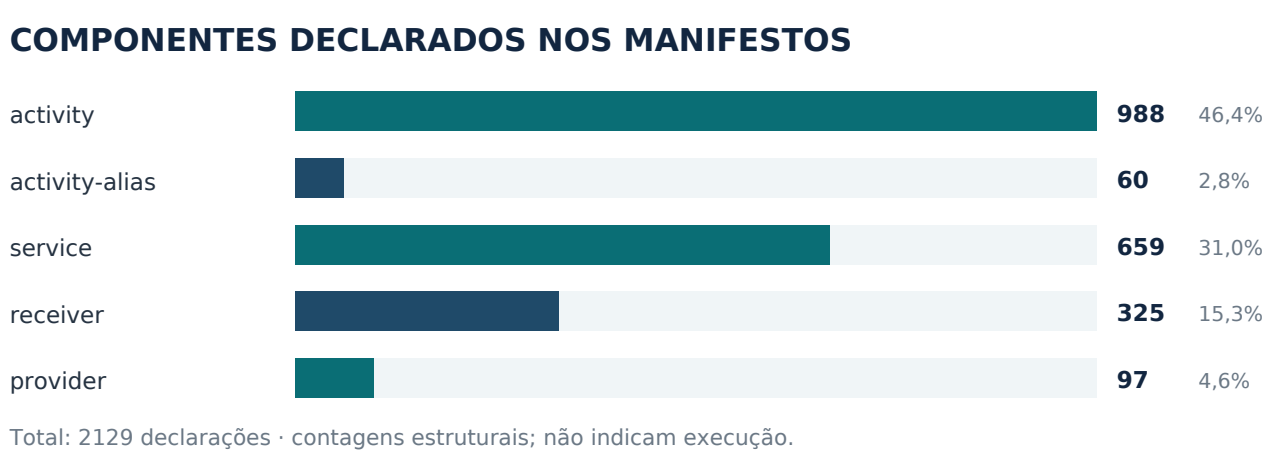


Figura 3. Contagens de declarações extraídas por aapt. A presença de um componente não comprova sua execução ou exposição a terceiros.

4.2 Privilégios e certificados

Os dois UOTA solicitam INTERNET, READ_PHONE_STATE, ACCESS_SUPERUSER e INSTALL_PACKAGES, além das permissões de rede e de uma permissão específica de cada aplicativo. Ambos compartilham android.uid.system/1000 no dumpsys. O contexto observado é privilegiado, mas UID 1000 não é UID 0. O comando sh iniciado pelo aplicativo herda seu contexto; a existência de ADB root não demonstra que todo subprocesso de aplicativo seja root.

Os certificados DER extraídos dos blocos v1 dos dois UOTA coincidem com o certificado extraído de framework-res.apk. A correspondência é reproduzível e consistente com a integração à plataforma; a extração não validou a assinatura criptográfica dos APKs nem os esquemas v2/v3. O campo apkSigningVersion do dumpsys é informação reportada pelo sistema, não uma verificação independente deste trabalho. O hash do certificado comum está no apêndice A.

A triagem completa registra metadados para os 94 pacotes; não significa reconstrução funcional de todos eles. Os marcadores “Desconhecido; revisar finalidade” do inventário inicial preservam essa distinção. A classificação detalhada de achados é apresentada na seção 10, sustentada pelos componentes efetivamente examinados.

5. Inicialização e provisionamento

Foram mapeadas as declarações dos 49 arquivos init, com serviço, executável, argumentos, usuário/grupo declarados, classes, flags e linhas de origem. Declaração não comprova execução de todas as ações. A análise também considera os imports de arquivos específicos do hardware.

Dos oito scripts referenciados, três foram obtidos e analisados:

Script	Comportamento estático	Condições/limites
precopy.sh	Copia /system/precopy/* para /data/app/, ajusta proprietário e cria marcador	Apenas se /data/system.notfirstrun.precopy não existir; não foi executado
preinstall.sh	Chama pm preinstall em /system/preinstall e /sdcard/preinstall; cria marcador; chama qw --daemon	As duas chamadas preinstall dependem de marcador ausente. qw fica fora do if. Implementação vendor de pm preinstall não auditada
log_bg.sh	Rotaciona logs em /data/anr, remove gerações antigas e captura kmsg/logcat	Serviço declarado disabled/oneshot; não foi ativado para teste

Evidências: precopy.sh, preinstall.sh e log_bg.sh.

O init.common declara os serviços precopy/preinstall como root e disabled/oneshot; o gatilho sys.bootstrap_service_started=1 inicia preinstall. Isso identifica um mecanismo de provisionamento, sem demonstrar que a condição ocorreu durante esta coleta, que os diretórios continham APKs ou que a implementação aceitou qualquer APK. A leitura não apagou marcadores nem simulou primeiro boot/reset.

Ausentes nos caminhos pesquisados: /sbin/init_parttion.sh, /system/bin/bootclone.sh, /system/bin/install-recovery.sh, /system/xbin/log_temperature.sh e /system/xbin/start_memtester.sh. O log registra “remote object does not exist” para cada um. Parte das referências é de recovery ou serviços desabilitados; não se conclui que o sistema esteja quebrado ou que os arquivos não existam em outra imagem/estado.

Um serviço adicional qw é declarado como /system/bin/qw --daemon, user root, class core, e getprop informa init.svc.qw=running. O binário e sua função ainda não foram analisados; o estado reportado não atribui comportamento de rede nem prova malícia. Essa é uma pendência para correlação dinâmica, com prioridade devido ao contexto privilegiado. Evidências: declaração qw e estado reportado qw.

Também foi identificado um init que declara suinit e su --daemon em post-fs-data; seu efeito efetivo não foi testado. Anexos: serviços init, ações/imports e scripts e ausências.

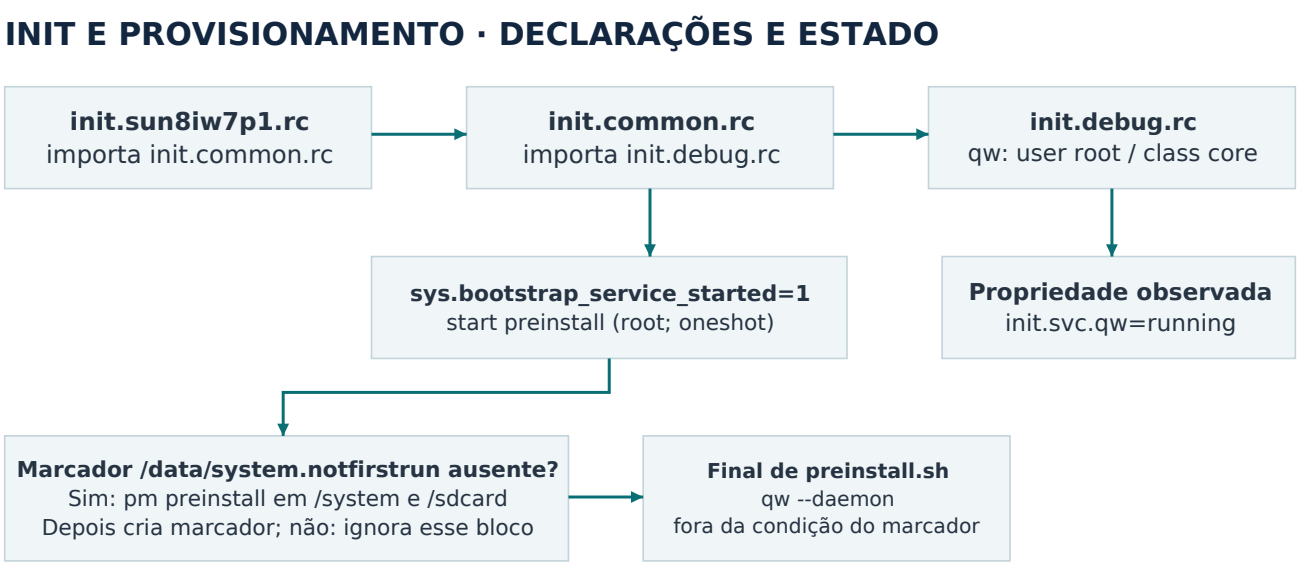


Figura 5. Encadeamento das declarações e scripts coletados. O marcador limita a pré-instalação; a chamada qw no final de preinstall.sh fica fora do if. O estado running é uma propriedade observada, não uma atribuição de finalidade.

A referência a /sdcard/preinstall merece atenção por envolver armazenamento externo em um serviço declarado root. A constatação é a chamada ao comando vendor pm preinstall. Não foram auditadas suas validações, os conteúdos desses diretórios ou a possibilidade de um usuário introduzir APKs aceitos por esse mecanismo. Não se afirma, portanto, uma exploração confirmada de instalação arbitrária.

6. Seleção dos aplicativos prioritários

A seleção combina impacto potencial e questões concretas do repasse dinâmico: o par UOTA por compartilhar UID de sistema e comunicar-se com o backend de atualização; qcode pela finalidade e gatilho desconhecidos; GetApps pelo tráfego observado e capacidade de gerenciar conteúdo. Todos os quatro APKs possuem classes.dex, mas a reconstrução útil varia conforme o componente.

Aplicativo	Versão / código	UID	Caminho	Motivo da análise
com.uv.droid.uota	1.0.3 / 10003	1000	/system/app/UOTA_aikePro103/	Atualização, privilégios e requisição de profile
com.uv.droid.uotaback	1.0.2 / 10002	1000	/system/app/UOTABack_aikePro102/	Segundo cliente OTA e manutenção do principal
com.qcode.ad_service_server	1.0.7777.976 / 976	10063	/data/app/...-1/	Serviço sem ícone e ausência de execução observada
com.simple.appstore.flymarket	2.0.3 / 2000399	10061	/data/app/...-1/	Loja GetApps e tráfego após interação

As permissões solicitadas são 7, 7, 29 e 13, respectivamente. Os componentes declarados são 7 no principal, 6 no backup, 4 no qcode e 14 na GetApps. Esses totais são cobertura estrutural, sem graduação automática de risco por quantidade.

7. Reconstrução técnica dos aplicativos

7.1 UOTA principal: profile, update e instalação

Os endpoints não são equivalentes: GET api/common/ota/profile/v1 retorna ConfigurationInfo (productKey, uuid, packages, properties). POST api/common/ota/update/v1 retorna List<UpdateInfo> com pacote, versão, método, URL e MD5. Endpoints: api/OtaApiService.java:57. O endereço padrão é HTTP: BuildConfig.java:15.

O método isConfigurationReady não foi reconstruído em Java legível. Sua saída com --comments-level debug contém o dump de instruções: consulta profile, atribui o result à configuração, exige productKey/uuid não vazios, salva JSON e timestamp e atualiza os parâmetros de consulta. Evidência do dump: fallback/OtaAutoCheck_uota_debug.java:420, atribuição em 429, validação a partir de 436 e atualização dos parâmetros em 513. Esses trechos são um dump produzido pelo JADX, não código fonte original nem Java recompilável. O resultado foi lido manualmente; o stub UnsupportedOperationException criado pelo decompilador não deve ser confundido com comportamento original do APK.

O encadeamento seguinte é visível no Java gerado:

1. O ciclo chama isConfigurationReady e isNeedToUpdate e executa as tarefas: service/OtaAutoCheck.java:184.

2. A resposta de update cria as tarefas: service/OtaAutoCheck.java:452.

3. URL, MD5 e método são copiados da resposta para a tarefa: service/OtaHelper.java:49.

4. A tarefa baixa pela URL recebida: service/OtaTaskSm.java:463.

5. Compara MD5 do arquivo com o fornecido na resposta; em divergência apaga o arquivo e marca falha: service/OtaTaskSm.java:500.

6. Download validado transita para instalação: service/OtaTaskSm.java:559.

7. Na API 24, a instalação silenciosa chama pm install -r -i <pacote_instalador> --user 0 <arquivo>: utils/InstallUtils.java:113. O executor abre sh: utils/ProcessUtils.java:50.

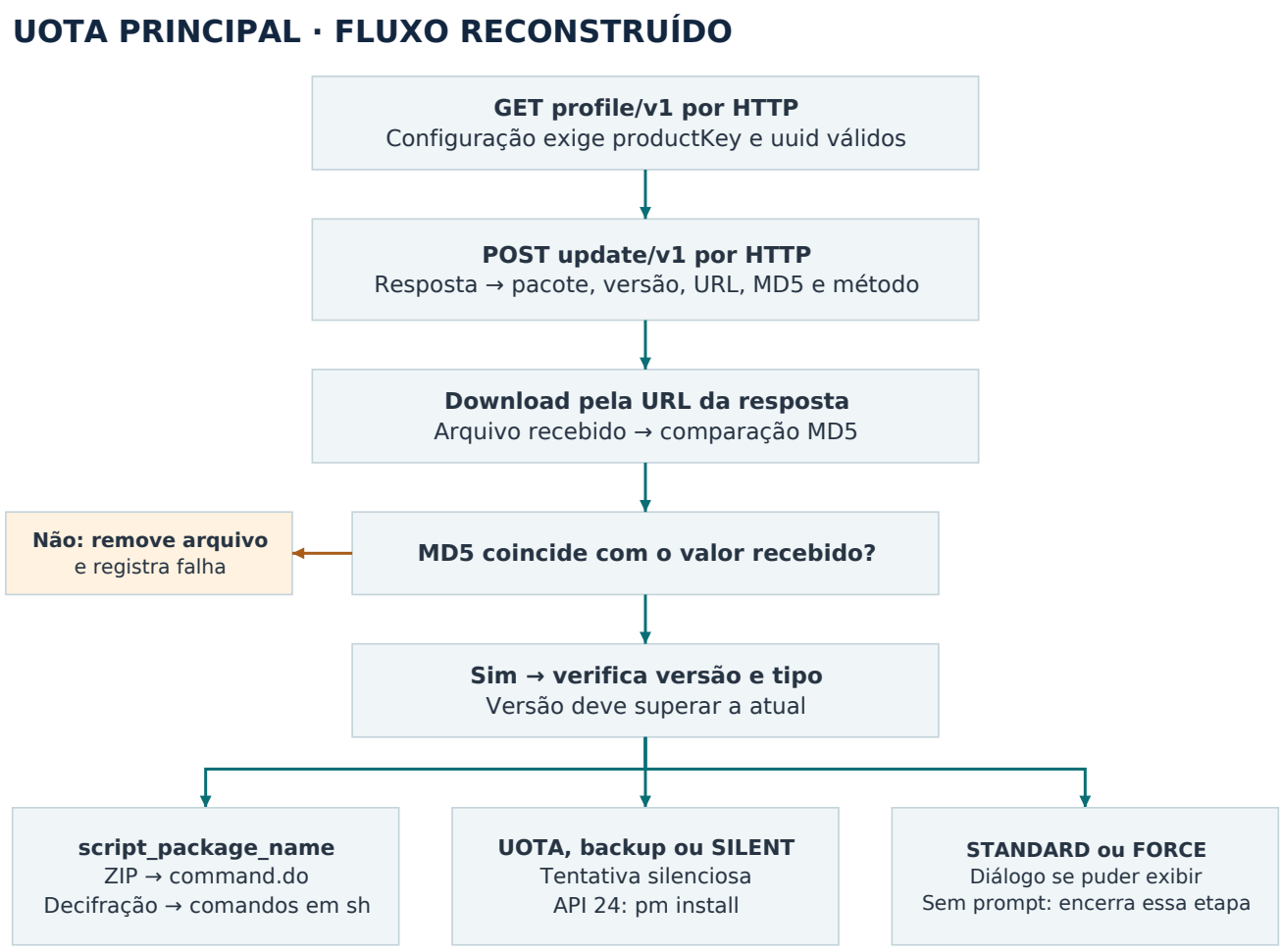


Figura 4. Fluxo reconstruído no UOTA principal. As decisões e rejeições são parte do resultado; não foi realizada instalação experimental.

Condições que limitam a conclusão: é necessário um profile válido e uma resposta update válida; a versão recebida precisa superar a local no teste do instalador; o download e o MD5 precisam passar. STANDARD e FORCE exibem diálogo na implementação lida; SILENT não exige esse diálogo. Atualizações dos próprios UOTA/backup entram no caminho silencioso independentemente desse método: service/OtaTaskSm.java:608. Se pm falhar, há tentativa de abrir o instalador por Intent; portanto, não afirmar que toda tentativa é silenciosa ou que todas as instalações necessariamente terão sucesso.

Verificação de autenticidade: foi encontrada comparação MD5, mas não uma assinatura independente da resposta/payload nos métodos que compõem esse fluxo. MD5 e URL vêm do mesmo canal HTTP: um hash fornecido junto com o arquivo por uma fonte não autenticada não autentica seu emissor. Isso sustenta risco de alteração da atualização por quem controlar esse canal. A instalação pelo Android é uma camada distinta; não foi demonstrado um bypass dos verificadores do sistema nem que qualquer APK arbitrário substituiria um pacote assinado existente.

O cliente também configura um HostnameVerifier que retorna true e um X509TrustManager com checkServerTrusted vazio: api/OtaApiService.java:65, utils/HttpUtils.java:23. Trocar apenas a URL para HTTPS não comprovaria correção dessa configuração.

Há ainda uma tarefa especial script_package_name: procura command.do no ZIP, decifra linhas com parâmetros AES constantes e passa os comandos a sh: service/OtaTaskSm.java:674. Não foram recebidos nem executados scripts. Esse é um caminho potencial de execução de comandos sob o contexto do aplicativo (UID 1000 reportado), não prova de execução efetiva com UID 0 ou de comando hostil.

Conclusão P1: a análise confirma uma cadeia estática de atualização e instalação silenciosa, com controles e condições identificados. A fragilidade do canal e da autenticidade sustenta a classe vulnerável/mal configurado. Instalação silenciosa é uma capacidade de atualizadores legítimos também: por si só não comprova malware. Para “indesejado comprovado”, ainda seria necessário demonstrar um conteúdo/ação incompatível efetivamente selecionado ou entregue, com contexto.

7.2 UOTA backup: manutenção cruzada e persistência

O principal inclui sempre os dois nomes UOTA na consulta, além dos pacotes recebidos no profile: service/OtaAutoCheck.java:428. O backup tem endpoints otabackup/profile e otabackup/update: api/OtaBackApiService.java:58. Seu pedido inclui o principal somente se isAppInstalled for verdadeiro: service/OtaAutoCheck.java:417. Seu estado Installing chama silentInstall apenas para com.uv.droid.uota: service/OtaTaskSm.java:471.

Isso confirma manutenção/atualização cruzada e explica a redundância como inferência de projeto, sem comprovar um mecanismo simétrico de reinstalação após remoção. No caminho analisado, o backup não solicita atualização do principal quando ele está ausente. A finalidade do MonitorReceiver do principal é emitir métricas de eventos de pacotes/IPC, e não reinstalar diretamente outro app: receiver/MonitorReceiver.java:19.

Ambos têm LoaderReceiver com boot/conectividade no manifesto; no código, o início do serviço depende de rede conectada. Principal: receiver/LoaderReceiver.java:16. Isso ajuda a delimitar os gatilhos do Grupo 2, sem garantir o estado de execução em toda situação.

7.3 Fontes de hora HTTP e NTP

Google/Baidu estão em WebTimeProvider de ambos os pacotes. A chamada é HttpURLConnection.getDate() e o valor positivo atualiza a fonte de hora do app: ntp/WebTimeProvider.java:20, ntp/WebTimeProvider.java:50, ntp/WebTimeProvider.java:50. Isso mostra consulta de hora via data HTTP; não sustenta atribuir essas conexões somente a uma checagem de conectividade. O código existe nos dois, mas o PID compartilhado/dados do Grupo 2 ainda não permitem dizer qual abriu cada conexão concreta observada.

A lista NTP tem os seis IPs do repasse, alternados com nomes pool.ntp.org, além de outros IPs e time.google.com: ntp/SntpTimeProvider.java:19, ntp/SntpTimeProvider.java:19. O cliente tenta as fontes até uma funcionar. TrueTime combina fontes local/NTP/servidor/web e agenda novas tentativas em caso de falha: ntp/TrueTime.java:30, ntp/TrueTime.java:60. O tempo é usado em timestamps/validade de caches de configuração e IP e em decisões de temporização. Não foi encontrada uma validação de assinatura de atualização baseada nessa hora no fluxo examinado.

Atenção ao nome enganoso SystemTimeProvider.setSystemTime: o método lido apenas marca inicialização se o valor estiver próximo do relógio local; não altera o relógio do Android: ntp/SystemTimeProvider.java:26.

A lista embutida possui 18 entradas: oito nomes pool.ntp.org regionais/globais, time.google.com e nove IPs literais. Os seis IPs destacados no repasse são 176.9.118.9, 61.216.153.106, 203.135.184.123, 108.59.2.24, 164.73.227.4 e 201.217.3.86; as três entradas IP adicionais são 200.189.40.8, 200.192.232.8 e 216.239.35.12. A ordem pode explicar tentativas repetidas quando as fontes anteriores falham, sem provar que toda a lista foi contatada em cada execução.

Interpretação: os métodos de hora esclarecem a finalidade estática das conexões, mas não atribuem cada socket a um dos dois processos que compartilham UID. O uso de HTTP sem autenticação como fonte de hora pode afetar timestamps e temporização do cliente. Não foi demonstrada manipulação do relógio, validação de assinatura dependente desse horário ou alteração do relógio global.

7.4 qcode: boot, serviços e exclusão potencial

O manifesto declara um BootCompleteReceiver, AdServer exportado (processo com.qcode.ad_service) e dois serviços não exportados (CornerAdService, WorkerAdService). Evidência: manifesto qcode. A application declara persistent=true; não presumir que isso torna efetivamente persistente um APK que o dump registra em /data/app sem a flag SYSTEM.

Ao receber boot, o receiver inicia AdServer; na API 24 chama startService: BootCompleteReceiver.java:21. O dump da coleta mostra stopped=true/notLaunched=true no usuário 0. Isso é compatível com a hipótese de exclusão da entrega normal de broadcasts a pacotes stopped, conforme documentação do Android. Hipótese: o estado do momento da coleta não prova o estado em cada experimento anterior. Não executar o receiver na unidade de referência para resolver a dúvida.

O receiver também inicia uma thread que chama deleteDir no armazenamento externo: BootCompleteReceiver.java:34. A rotina percorre diretórios, exclui arquivos maiores que 10 MiB e preserva três nomes titan específicos: utils/DiskIOUtils.java:25, utils/DiskIOUtils.java:597. Trata-se de capacidade potencial de exclusão; o resultado depende do gatilho, permissões e arquivos acessíveis. Não foi executada nem foi comprovada perda de arquivos. Por isso, qualquer teste desse gatilho deve usar uma unidade experimental sem dados pessoais/institucionais, com registro antes/depois.

O manifesto solicita 29 permissões, incluindo armazenamento, câmera, microfone e instalação de pacotes. O bloco de permissões de instalação no dumpsys não registra essas capacidades como concedidas; não há bloco de runtime permissions nessa entrada. Essa leitura não autoriza inferir que todas as permissões solicitadas estejam disponíveis. O receiver não contém uma checagem de permissão antes da chamada deleteDir no trecho examinado, mas o acesso aos arquivos continua sujeito aos controles efetivos do sistema.

Os nomes preservados pela rotina são titan-com.qcode.ad_service_server, titan-com.qcode.rn.launcher e titan-io.titannetwork.service. A comparação é feita pelo nome em cada nível da recursão. Além de excluir arquivos acima de 10485760 bytes, a rotina tenta remover o diretório visitado ao terminar; o sucesso de cada delete não é conferido nesse método. Esse comportamento merece investigação por seu possível impacto sobre mídia local, sem demonstrar dano ocorrido.



Figura 6. Capacidade estática do receiver; o estado stopped/notLaunched e as permissões limitam a atribuição de execução e de efeitos reais.

7.5 GetApps: permissões e fronteira de análise

O manifesto declara RequestService, AllAppsRequestService, CleanerService, CoreService, FileProvider e componentes do downloader. Declara DOWNLOAD_WITHOUT_NOTIFICATION e KILL_BACKGROUND_PROCESSES, mas não INSTALL_PACKAGES. A lista solicitada está em permissoes_solicitadas.csv; o dump reporta ambas as primeiras concedidas (linhas 9165 e 9170). Declarar/ter permissão não prova seu uso.

A application é com.stub.StubApp, e o DEX disponível foi decompilado em apenas 10 arquivos Java, essencialmente camada de carregamento/recursos, sem os métodos da loja. Há chamadas nativas e carga de bibliotecas jiagu: com/stub/StubApp.java:683, com/stub/StubApp.java:708. O APK contém assets/libjiagu.so e assets/libjiagu_x86.so; o inventário passou a incluir também essas bibliotecas em assets. Elas não foram carregadas/executadas.

Resultado inconclusivo: não foi possível determinar estaticamente o caminho de instalação, o uso efetivo das duas permissões ou a construção do endpoint appstore.bb20.site:2086. O indicador Bugly existe nos arquivos, mas não prova a finalidade completa do SDK. Não interpretar a ausência de classes no JADX como inexistência de comportamento. Próxima investigação: análise estática da camada nativa e/ou coleta controlada dos artefatos carregados, em unidade experimental separada, preservando a referência. Nenhuma dessas etapas foi realizada aqui.

7.6 Modo FEL: resultado fora da aquisição lógica

Não foi realizado teste de sunxi-fel, leitura ou gravação de flash. O VID/PID temporário de boot reportado pelo Grupo 2 e o funcionamento de ADB são observações distintas. Não se conclui que a interface FEL permita leitura/gravação nesta unidade. Essa pergunta fica não testada, sem impedir o registro dos resultados estáticos produzidos ou exigir modificação da referência.

8. URLs, indicadores e cobertura da pesquisa

A pesquisa guiada abrangeu 31 indicadores recebidos e 9 acrescentados, com 342 registros indicador/arquivo. Os endpoints Retrofit estavam sem a barra inicial na string relativa, explicando negativos da primeira busca literal.

Foi também feita extração ampla de candidatos de URLs em todos os APKs e arquivos relevantes (DEX, recursos, assets, bibliotecas, ODEX/OAT, init e build.prop). Foram registradas 27198 ocorrências candidatas, com arquivo/membro, offset e codificação, sem avisos de leitura/tamanho nessa execução. Isso inclui esquemas XML, exemplos, documentação e SDKs: não são 27198 conexões nem achados de risco. As strings não demonstram chamada efetiva ou resolução do host. URLs montadas, criptografadas ou obtidas em execução podem escapar da pesquisa.

Indicadores prioritários: endpoints OTA/otabackup, o canal de métricas appmetrices.uota.xyz:2095/els/postData2, fontes NTP/web e a modalidade de comandos. O domínio vxmfc.com aparece como Referer do downloader, sem comprovar conexão a essa origem. O domínio da loja permanece como observação dinâmica do Grupo 2, sem construção interna reconstruída nesta análise.

Anexos: indicadores guiados, candidatos de URLs e limitações da pesquisa. Resultados negativos registrados incluem os cinco caminhos .sh ausentes, a lógica protegida não reconstruída e o teste FEL não realizado.

8.1 Indicadores selecionados para validação

Indicador	Origem estática	Interpretação e limite
uota.uota.xyz:2082 e caminhos profile/update	BuildConfig e interfaces Retrofit dos dois UOTA	Canal OTA HTTP; profile e update têm funções diferentes
appmetrices.uota.xyz:2095/els/postData2	MyApplication do principal	Endpoint de métricas configurado; tráfego não comprovado nesta etapa
www.google.com / www.baidu.com	WebTimeProvider de ambos	Fonte de hora pelo cabeçalho Date; não apenas teste de conectividade
Seis IPs NTP do repasse e demais entradas	SntpTimeProvider de ambos	Lista de fontes de hora e ordem de tentativa
http://vxmfc.com	Referer em OtaHelper	Cabeçalho configurado; não prova conexão ao host
script_package_name / command.do	OtaTaskSm e constantes	Modalidade de comandos; nenhum conteúdo recebido/executado
BootCompleteReceiver / AdServer	Manifesto e código qcode	Gatilho de boot e serviço; execução condicionada ao estado da aplicação
/sdcard/preinstall / qw --daemon	Script preinstall e init	Provisionamento e daemon desconhecido; validações/finalidade abertas
appstore.bb20.site:2086	Observação dinâmica do Grupo 2	Não reconstruído na camada funcional protegida da GetApps

Os indicadores são candidatos para observação/correlação. Sua inclusão não constitui lista de domínios comprovadamente maliciosos. Classificação e medidas dependem da função e da política de uso, não de sufixo de domínio, país de hospedagem ou nome do pacote.

9. Correlação com evidências dinâmicas recebidas

O texto do relatório do Grupo 2 e os arquivos de evidência recebidos foram usados como contexto, com atribuição explícita da origem. Os experimentos de rede não foram repetidos pelo Grupo 1. A análise estática amplia a explicação de capacidades que uma captura limitada ao estabelecimento de conexões não revela.

Observação ou pergunta dinâmica	Evidência estática do Grupo 1	Resultado da correlação
E3: GET profile por HTTP no boot, com identificadores persistentes	BuildConfig, Retrofit e leitura do profile	Compatível com o cliente identificado; a resposta 404 vazia não permite observar update/download
E2/E3: os dois UOTA aparecem no boot	LoaderReceiver e manutenção cruzada	Gatilhos e papéis identificados; não comprova reinstalação após remoção
P1: o que acontece com a resposta do servidor?	ConfigurationInfo → UpdateInfo → tarefa → download → MD5 → decisão	Fluxo reconstruído; instalação silenciosa e comandos são capacidades condicionais, sem execução experimental
E5: Google/Baidu atribuídos ao par UOTA; finalidade de conectividade inferida	WebTimeProvider usa getDate() nos dois	Finalidade de obtenção de hora esclarecida; socket individual continua sem atribuição definitiva
E6: seis IPs NTP e tentativas ordenadas	Lista SntpTimeProvider e uso por TrueTime	Destinos e temporização têm explicação no código; não são prova de canal de comando
qcode não aparece em execução nos experimentos	Receiver existe; snapshot posterior stopped/notLaunched	Hipótese compatível; não demonstra o estado em cada experimento anterior
E4: GetApps tenta acessar backend e Bugly	Manifesto, indicador Bugly e carregamento jiagu	Correlação parcial; construção do backend e caminho de instalação permanecem inconclusivos
qw não teve finalidade estabelecida no material recebido	Serviço declarado root e init.svc.qw=running	Novo ponto de investigação; sem atribuição de tráfego ao binário

A análise corrige duas inferências que o código não sustenta: Google/Baidu não devem ser descritos somente como checagem de conectividade; a presença de dois atualizadores não demonstra uma rotina simétrica de reinstalação caso um deles seja removido. Também não se toma a presença em Cloudflare, um domínio .xyz ou o nome GetApps como prova autônoma de malícia.

10. Matriz de achados e classificação

A classificação segue a seção 11 da proposta. Nomes, origem, permissões, strings, proteção de código e conexão isolada não comprovam malícia. A prioridade abaixo é qualitativa, relativa à investigação/reaproveitamento, sem pontuação CVSS ou CVE demonstrada. Classe e prioridade não são a mesma informação.

ID	Achado	Classificação	Prioridade/limite
G1-01	Versão declarada inconsistente	Vulnerável/mal configurado	Média
G1-02	Perfil de depuração/root e SELinux permissivo	Vulnerável/mal configurado	Alta se depuração acessível
G1-03	Atualização HTTP e instalação silenciosa condicional	Vulnerável/mal configurado	Alta
G1-04	Validação TLS permissiva no cliente UOTA	Vulnerável/mal configurado	Alta se HTTPS usado
G1-05	Modalidade de atualização com comandos	Potencialmente inseguro/suspeito	Alta potencial
G1-06	Limpeza de arquivos grandes no receiver qcode	Potencialmente inseguro/suspeito	Média potencial
G1-07	Pré-instalação com diretório em armazenamento externo	Potencialmente inseguro/suspeito	A determinar
G1-08	Serviço qw sem finalidade determinada	Desconhecido	A determinar; prioridade alta
G1-09	Lógica GetApps protegida por camada nativa	Desconhecido	A determinar

A matriz estruturada, com evidências e recomendações, está em ACHADOS.csv e é complementada pelas análises das seções 5–8 deste relatório. Não foi classificado qualquer componente como malicioso/indesejado comprovado. A leitura demonstrou capacidades e configurações; intenção e payloads hostis efetivos não foram demonstrados.

O inventário inicial contém marcadores “Desconhecido; revisar finalidade” para pacotes ainda não interpretados funcionalmente. Isso é cobertura de inventário, não uma investigação profunda de todos os 94 APKs. A seleção dos quatro prioritários veio dos comportamentos/perguntas do Grupo 2, mantendo a cobertura estrutural dos demais pacotes.

10.1 Alcance e consequência de cada achado

ID	Consequência potencial	Evidência decisiva	Condição ainda não demonstrada
G1-01	Seleção incorreta de requisitos/compatibilidade por confiar no rótulo	Release 11.1 em conflito com SDK 24/fingerprint 7.0	Intenção da alteração e RAM física correta

ID	Consequência potencial	Evidência decisiva	Condição ainda não demonstrada
G1-02	Acesso privilegiado e controles reduzidos na interface de depuração	Shell UID 0 e SELinux Permissive; propriedades de debug	Exposição remota atual ou exploração externa
G1-03	Alteração do canal de atualização; instalação definida por conteúdo remoto	HTTP + URL/MD5 remotos + fluxo de instalação	Payload hostil e sucesso de instalação/arbitrário bypass
G1-04	Aceitação de servidor/certificado não autenticado pelo cliente	HostnameVerifier true e checkServerTrusted vazio	Exploração TLS; endpoint padrão usa HTTP
G1-05	Execução de comandos entregues pelo mecanismo de atualização	command.do → decifração → sh	Recebimento/seleção e execução de comando hostil
G1-06	Remoção de mídia local acessível no armazenamento externo	Receiver → deleteDir; limiar >10 MiB	Gatilho, permissões efetivas, arquivos acessíveis e dano
G1-07	Pré-instalação baseada em diretório externo no provisionamento	pm preinstall /sdcard/preinstall, init root	Conteúdo, controle do diretório e validações do vendor
G1-08	Risco não quantificado de daemon privilegiado desconhecido	Serviço qw root e propriedade running	Função, dependências e comportamento de rede
G1-09	Capacidade funcional da loja não completamente auditada	StubApp/jiagu e manifesto disponíveis	Fluxos internos de download, instalação e limpeza

Os quatro primeiros achados descrevem configurações verificáveis no escopo analisado. Os três seguintes registram capacidades que justificam investigação por seu possível impacto. Os dois últimos preservam componentes cuja função/implementação não foi determinada. A tabela evita transformar potencial em ocorrência ou desconhecimento em acusação.

11. Recomendações decorrentes da análise

Prioridade	Requisito proposto	Evidência que motiva	Como verificar em trabalho posterior
Alta	Canal de atualização autenticado e política controlada de instalação	G1-03 e G1-04	Confirmar validação de identidade/integridade e rejeição de respostas não autorizadas
Alta	Restrição de depuração e revisão do perfil de privilégios	G1-02	Conferir estado efetivo de ADB, autenticação e controles de acesso
Alta de investigação	Determinar finalidade/dependências de qw e limitar comandos remotos	G1-05 e G1-08	Auditoria do binário e observação de processo/sockets em ambiente experimental
Média de investigação	Examinar qcode e política de preservação de mídia	G1-06	Teste isolado com arquivos sintéticos e inventário antes/depois
A determinar	Auditar validações de pré-instalação e camada nativa GetApps	G1-07 e G1-09	Revisão da implementação vendor e dos artefatos protegidos
Transversal	Definir componentes necessários à finalidade educacional	Inventário completo e dependências	Testes de interface, HDMI, áudio, rede e aplicações selecionadas

As recomendações não são medidas implementadas ou validadas pelo Grupo 1. Nenhum pacote foi declarado removível apenas por seu nome. Framework, interface, launcher, configurações e componentes de mídia/rede exigem avaliação funcional de dependências antes de qualquer alteração. A análise estática fundamenta os requisitos de controle; não certifica a estabilidade após desativação de componentes.

12. Limitações e resultados inconclusivos

A amostra única e compartilhada não representa o lote. Não há dump integral, verificação completa de assinaturas, reconstrução de todos os apps/bibliotecas, execução de atualização nem auditoria nativa de qw/GetApps. Evidências de configuração foram lidas do sistema em execução e podem conter metadados incorretos.

A verificação do material recebido do Grupo 2 detectou divergência somente em coletado_em.txt na pasta etapa0 (os outros seis arquivos daquela lista passaram). Registrado: 8db169b57885568a835e3d863cf21fc5411a2bf541b494c246ffb1cd127f8b2d; calculado: d0dfea45e133377d3a3b90b48b534d130d6798c40be0f09c32fafb82cf5f16c3. A causa é desconhecida; os originais foram preservados. Não significa adulteração intencional, nem invalida automaticamente as demais evidências, mas a data desse arquivo precisa de confirmação. A verificação referida abrangeu etapa0, não todo acervo dinâmico. Esta divergência não se refere às novas coletas do Grupo 1.

Outros resultados negativos estão documentados: cinco scripts não existem nos caminhos montados pesquisados; o método de profile não gerou Java legível e exigiu dump de instruções; o domínio da GetApps não foi reconstruído na lógica protegida; e FEL não foi testado. Cada negativo descreve o procedimento realizado e sua fronteira, sem extrapolar para ausência universal.

Não foi produzida análise manual de finalidade para todos os 94 pacotes nem verificação integral de assinaturas. As bibliotecas foram relacionadas aos APKs, mas a lógica nativa não foi completamente auditada. A pesquisa por strings pode perder URLs construídas, criptografadas ou obtidas em execução. Nenhum teste de exploração, instalação remota, perda de arquivos ou comparação após hardening foi realizado nesta etapa.

13. Conclusão

O Grupo 1 produziu os cinco tipos de entregável previstos na seção 8.3: inventário estruturado, mapa de permissões/inicialização, indicadores para validação dinâmica, classificação preliminar baseada em evidências e relatório técnico. O inventário cobre a composição registrada, e a análise aprofundada reconstrói mecanismos concretos de atualização, hora, inicialização e provisionamento. As aquisições e seus derivados possuem referências de integridade e reprodução; a divergência entre a data informada pelos autores e o relógio do host está documentada na seção 3.2.

A unidade examinada apresenta inconsistência de versão declarada, perfil de depuração privilegiado e mecanismos de atualização com canal sem autenticação adequada identificada no fluxo. Foram também registradas capacidades potencialmente inseguras e componentes desconhecidos. A análise não comprovou conteúdo malicioso nem execução de ações hostis; não oferece fundamento para atribuir esse comportamento a todo o lote.

Para eventual reaproveitamento educacional, os resultados sustentam revisão de depuração/privilégios, controle de atualização e investigação dos componentes desconhecidos. A adequação operacional depende de validação dos controles e das funções essenciais. O produto desta etapa é uma base estática documentada para decisões técnicas, com conclusões proporcionais às evidências.

Apêndice A — Catálogo de evidências e hashes representativos

A.1 Convenção de caminhos

Neste catálogo, C representa coleta/20261012_204535_l0sQMz/, I representa coleta_complementar/20261012_225942_ez2GjA/, A representa analise/20261012_204535_l0sQMz/ e E representa entrega/grupo1/. Os anexos são parte do conjunto documental; o PDF resume os resultados e não contém cópia integral dos APKs.

Código	Arquivo ou conjunto	Papel na análise
EV-01	C/sistema/getprop.txt e kernel_cpu.txt	Versão, UID, SELinux, build e estado qw
EV-02	C/sistema/dumpsys_package.txt	UIDs, estados, flags e permissões reportadas
EV-03	A/inventario_apks.csv e certificados_v1.csv	Metadados, SHA-256 e certificados extraídos
EV-04	A/permissoes_solicitadas.csv e componentes_manifesto.csv	Manifestos e filtros com linhas de origem
EV-05	A/bibliotecas_nativas.csv	Relação de entradas .so e APKs
EV-06	C/fontes/ e A/fallback/	Reconstrução parcial e dumps revisados
EV-07	C/sistema/init/ e E/anexos/servicos_init.csv, acoes_init.csv	Declarações init e origem por arquivo/linha
EV-08	I/scripts/, resultados.tsv e logs/pull.log	Scripts complementares e evidências de ausência
EV-09	C/indicadores.csv e E/anexos/urls_estaticas.csv	Pesquisa guiada e candidatos com offsets
EV-10	E/ACHADOS.csv	Classificação, prioridade, evidência e recomendação
EV-11	evidencias/ e referencias/grupo2/relatorio_grupo2.md	Contexto dinâmico recebido, com limitações de integridade

A.2 APKs prioritários

com.uv.droid.uota Caminho: /system/app/UOTA_aikePro103/UOTA_aikePro103.apk SHA-256: af65c0e1adf6e03a7b013006a8b6bb7afdcea56a0e84a2e73fa53fd17702200d

com.uv.droid.uotaback Caminho: /system/app/UOTABack_aikePro102/UOTABack_aikePro102.apk SHA-256: 799b4cd069a975f11772dcffd83902c2a8381866c2762fa098a7b36c15a54fe2

com.qcode.ad_service_server Caminho: /data/app/com.qcode.ad_service_server-1/base.apk SHA-256: 009fc1299d0a1c3c72017983927273aca9620c0de7ec31babec6ef128eb35fbc

com.simple.appstore.flymarket Caminho: /data/app/com.simple.appstore.flymarket-1/base.apk SHA-256: d8de40b3eb7ff342b0b554cae70946387c2909a0eb4e1e92f9e7a1f3773f6a23

O certificado DER comum aos dois UOTA e ao framework-res possui SHA-256 c8a2e9bccf597c2fb6dc66bee293fc13f2fc47ec77bc6b2b0d52c11f51192ab8. Esse valor identifica o certificado extraído; não constitui validação das assinaturas dos arquivos APK.

A.3 Referências de código decisivas

Componente	Arquivo relativo às fontes do pacote	Linhas da execução documentada	Resultado sustentado
UOTA	BuildConfig.java; api/OtaApiService.java	15; 57–65	URL HTTP, endpoints e hostname permissivo
UOTA	service/OtaAutoCheck.java	184, 428, 452	Ciclo, seleção de pacotes e tarefas
UOTA	service/OtaHelper.java	49, 74	Dados remotos da tarefa e Referer
UOTA	service/OtaTaskSm.java	463, 500, 559, 608, 674	Download, MD5, instalação e comandos
UOTA	utils/InstallUtils.java; ProcessUtils.java	113; 50	pm install na API 24 e executor sh
UOTA	utils/HttpUtils.java	23	TrustManager permissivo
UOTA/backup	ntp/WebTimeProvider.java; SntpTimeProvider.java	20, 50; 19	Fontes HTTP/NTP
UOTA	ntp/TrueTime.java; SystemTimeProvider.java	30, 60; 26	Uso e limite da hora obtida
Backup	service/OtaAutoCheck.java; OtaTaskSm.java	417; 471	Principal presente e caminho silencioso
qcode	BootCompleteReceiver.java; utils/DiskIOUtils.java	21, 34; 25, 597	Serviço de boot e limpeza potencial
GetApps	com/stub/StubApp.java	683, 708	Camada de carregamento nativo
Dump UOTA	A/fallback/OtaAutoCheck_uota_debug.java	420–513	Profile, validação e armazenamento de configuração

Apêndice B — Reprodução e organização dos anexos

B.1 Conferência dos conjuntos preservados

Executar a partir da raiz do projeto. Os comandos abaixo verificam os arquivos registrados, sem alterar a box ou refazer a aquisição.

```
(cd coleta/20261012_204535_l0sQMz && sha256sum --quiet -c SHA256SUMS)
(cd coleta_complementar/20261012_225942_ez2GjA && sha256sum --quiet -c SHA256SUMS)
sha256sum --quiet -c analise/20261012_204535_l0sQMz/SHA256SUMS_DERIVADOS
sha256sum --quiet -c entrega/grupo1/SHA256SUMS_ENTREGA
```

Código zero indica correspondência dos hashes verificados. As listas dos arquivos brutos são relativas à respectiva pasta de coleta; as listas dos derivados e da entrega são relativas à raiz do projeto. Revisões documentais têm nova lista de hashes; a versão anterior permanece arquivada em .historico/antes_limpeza_20261013.tar.gz.

B.2 Procedimentos disponíveis

Script	Procedimento
01_conectar_box.sh	Identificação e confirmação de acesso à unidade
02_coletar_firmware.sh --tudo	Nova aquisição lógica com diretório próprio
03_procurar_indicadores.py	Busca dos indicadores configurados
04_manifestos.sh	Extração aapt dos manifestos
05_decompilar.sh	Reconstrução parcial documentada e logs
06_inventario_inicial.py	Metadados, hashes, certificados e bibliotecas
07_mapear_manifestos.py	Permissões/componentes com filtros e origem
08_mapear_init.py	Serviços, ações/imports e referências a scripts
09_coletar_scripts_init.sh	Aquisição complementar dos caminhos referenciados
10_extrair_urls.py	Candidatos de URLs com offsets e codificação

A aquisição exige acesso USB/ADB; a análise offline utiliza os arquivos já preservados. Scripts de análise podem substituir derivados e mudar suas linhas/hashes. O script 05 retorna 2 quando detecta saída parcial; isso não significa inutilidade de todas as fontes. O dump de profile foi gerado por execução específica do JADX com single-class e comentários debug, preservada em A/fallback/. Não é necessário repetir a coleta para avaliar os resultados existentes.

Apêndice C — Entregáveis do Grupo 1

Entregável da seção 8.3	Resultado deste trabalho	Localização
Inventário estruturado dos APKs e componentes	94 APKs com metadados/hashes; 94 manifestos; bibliotecas e certificados extraídos	Seção 4 e EV-03 a EV-05
Mapa de permissões e persistência/inicialização	1272 permissões; componentes/filtros; 71 serviços e 793 ações/imports; scripts analisados/ausentes	Seções 4–5 e EV-04, EV-07, EV-08
Indicadores para validação dinâmica	Endpoints, fontes de hora, modalidade de comandos, gatilhos e componentes desconhecidos	Seção 8 e EV-09; respostas P1–P7
Classificação preliminar baseada em evidências	Nove achados com classe, prioridade, condições e limites	Seção 10 e EV-10
Relatório técnico de análise estática	Método, resultados, correlação recebida, recomendações, negativos e reprodução	Este documento e anexos

Amostra única, ausência de identificação física detalhada e uso compartilhado são limites expressos na seção 2. A classificação da GetApps permanece inconclusiva; FEL não foi testado. O relatório não exige que toda capacidade potencial seja exercitada para ser documentada, nem preenche lacunas com atribuição indevida de malware.

Apêndice D — Referências

[R1] Android Developers. https://developer.android.com/guide/topics/manifest/uses-sdk-element — Elemento uses-sdk e relação entre versões e níveis de API. Consultado para interpretar API 24 / Android 7.0.

[R2] Android Developers. https://developer.android.com/about/versions/android-3.1#launchcontrols — Android 3.1: controles de lançamento e estado stopped. Consultado para delimitar a hipótese de entrega de broadcasts ao qcode.